

Atividade 2 - Transformações no Domínio Espacial e da Frequência

Aluno: Vitor Fontenele - 1700778

Imports

```
In [1]: import numpy as np
import cv2
import matplotlib.pyplot as plt
from pathlib import Path
import os
```

Setup

```
In [2]: plt.rcParams['figure.figsize'] = (12, 8)
plt.rcParams['image.cmap'] = 'gray'

IMG_PATH = Path('gargantua.png')
OUT_DIR = Path('outputs')

def load_image(path=IMG_PATH):
    path = Path(path)
    if not path.exists():
        raise FileNotFoundError(f'Imagem não encontrada: {path.resolve()}')
    bgr = cv2.imread(str(path), cv2.IMREAD_UNCHANGED)
    if bgr is None:
        raise ValueError(f'Não foi possível ler a imagem: {path.resolve()}')
    if bgr.ndim == 2:
        return bgr
    return cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)

def to_gray(img):
    if img.ndim == 2: return img
    return (0.299 * img[:, :, 0] + 0.587 * img[:, :, 1] + 0.114 * img[:, :, 2])

def save_image(filename, img, out_dir=OUT_DIR):
    out_dir = Path(out_dir)
    out_dir.mkdir(parents=True, exist_ok=True)
    out_path = out_dir / filename
    arr = np.clip(img, 0, 255).astype(np.uint8)
    if arr.ndim == 3:
        arr = cv2.cvtColor(arr, cv2.COLOR_RGB2BGR)
    cv2.imwrite(str(out_path), arr)
    return out_path

def show_grid(images, titles, cols=3):
    n = len(images)
    rows = (n + cols - 1) // cols

    _, axes = plt.subplots(rows, cols, figsize=(cols * 5, rows * 5))
    axes = axes.flatten() if n > 1 else [axes]
```

```

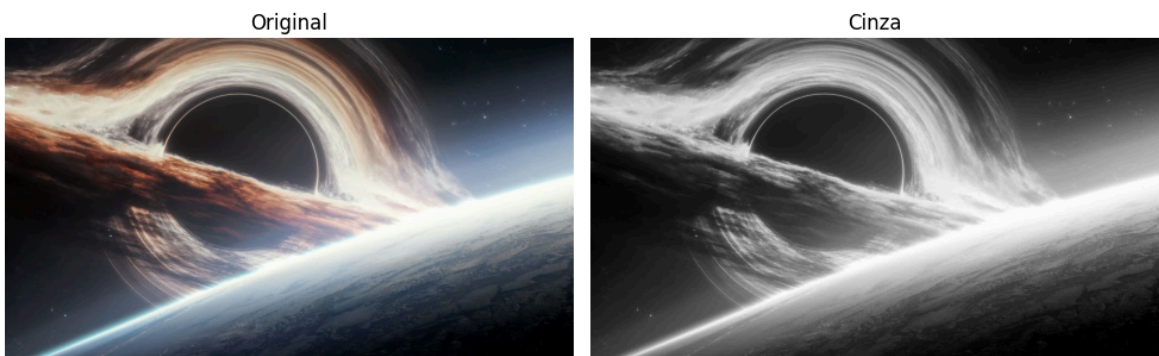
for i in range(len(axes)):
    if i < n:
        axes[i].imshow(images[i], vmin=0, vmax=255)
        axes[i].set_title(titles[i])
        axes[i].axis('off')

plt.tight_layout()
plt.show()

def initialize():
    img = load_image()
    img_gray = to_gray(img)
    save_image('gargantua_gray.png', img_gray, '.')
    show_grid([img, img_gray], ['Original', 'Cinza'], cols=2)

initialize()

```



Questão 1 - Convolução

Preparação para Convolução: Definição de filtros e função

```

In [3]: kernels = {
    "h1": np.array([
        [ 0,  0, -1,  0,  0],
        [ 0, -1, -2, -1,  0],
        [-1, -2, 16, -2, -1],
        [ 0, -1, -2, -1,  0],
        [ 0,  0, -1,  0,  0]
    ], dtype=np.float32),

    "h2": (1/256) * np.array([
        [1,  4,  6,  4,  1],
        [4, 16, 24, 16,  4],
        [6, 24, 36, 24,  6],
        [4, 16, 24, 16,  4],
        [1,  4,  6,  4,  1]
    ], dtype=np.float32),

    "h3": np.array([
        [-1,  0,  1],
        [-2,  0,  2],
        [-1,  0,  1]
    ], dtype=np.float32),

    "h4": np.array([

```

```

        [-1, -2, -1],
        [ 0,  0,  0],
        [ 1,  2,  1]
    ], dtype=np.float32),

    "h5": np.array([
        [-1, -1, -1],
        [-1,  8, -1],
        [-1, -1, -1]
    ], dtype=np.float32),

    "h6": (1/9) * np.array([
        [1, 1, 1],
        [1, 1, 1],
        [1, 1, 1]
    ], dtype=np.float32),

    "h7": np.array([
        [-1, -1,  2],
        [-1,  2, -1],
        [ 2, -1, -1]
    ], dtype=np.float32),

    "h8": np.array([
        [ 2, -1, -1],
        [-1,  2, -1],
        [-1, -1,  2]
    ], dtype=np.float32),

    "h9": (1/9) * np.eye(9),

    "h10": (1/8) * np.array([
        [-1, -1, -1, -1, -1],
        [-1,  2,  2,  2, -1],
        [-1,  2,  8,  2, -1],
        [-1,  2,  2,  2, -1],
        [-1, -1, -1, -1, -1]
    ], dtype=np.float32),

    "h11": np.array([
        [-1, -1,  0],
        [-1,  0,  1],
        [ 0,  1,  1]
    ], dtype=np.float32)
}

def convolution(imagem, filtro):
    i_h, i_w = imagem.shape
    f_h, f_w = filtro.shape

    pad_h, pad_w = f_h // 2, f_w // 2

    imagem_pad = np.pad(imagem, ((pad_h, pad_h), (pad_w, pad_w)), mode='c')

    saida = np.zeros_like(imagem, dtype=np.float32)

    for i in range(i_h):
        for j in range(i_w):
            regioao = imagem_pad[i:i+f_h, j:j+f_w]

```

```
saida[i, j] = np.sum(regiao * filtro)

return saida
```

Rodando

```
In [4]: img = load_image('gargantua_gray.png')
resultados = []
titulos = []

for nome, filtro in kernels.items():
    print(f"Aplicando {nome}...")

    img_filtrada = convolution(img, filtro)

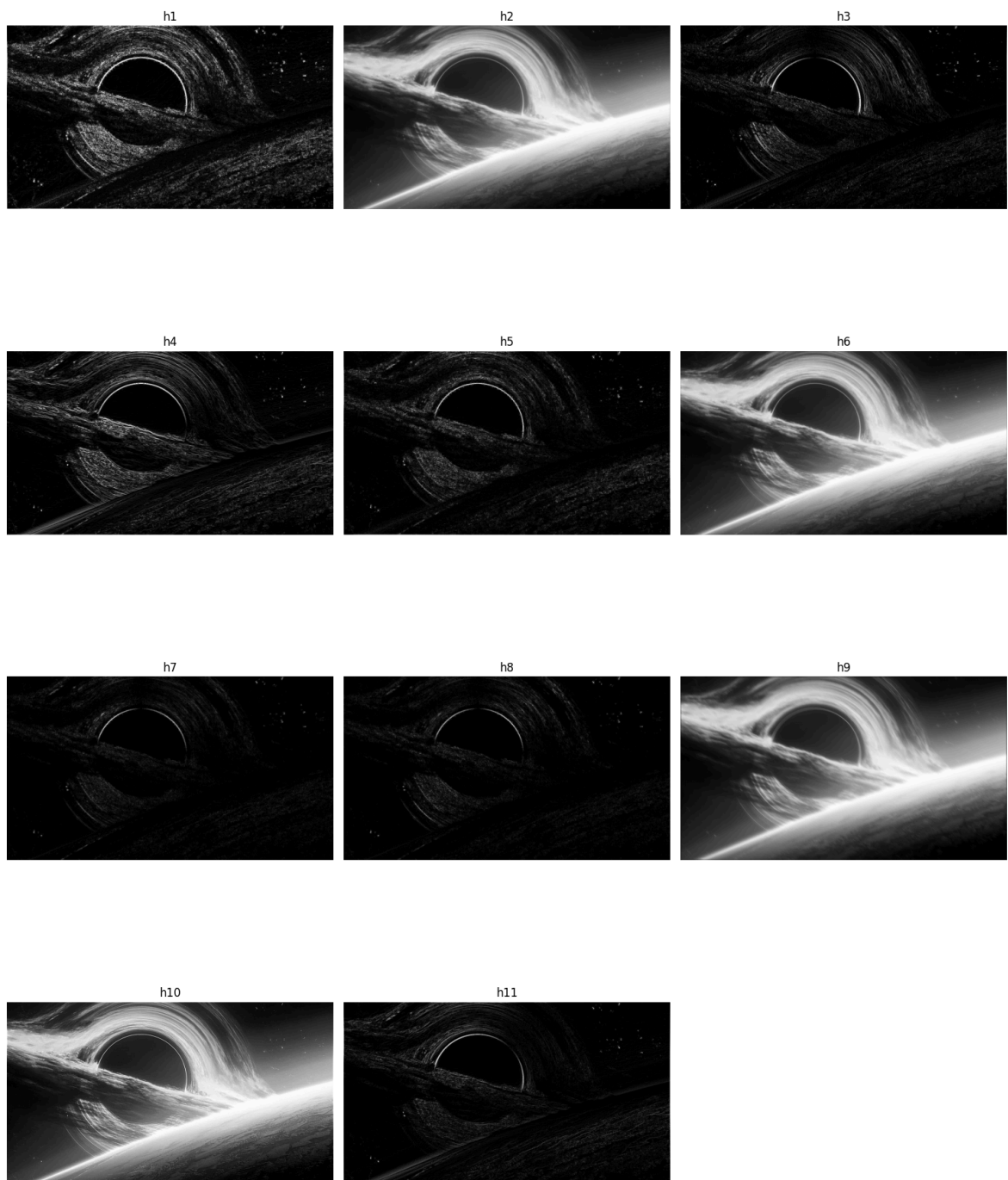
    img_visualizacao = np.abs(img_filtrada)

    resultados.append(img_visualizacao)
    titulos.append(nome)

    save_image(f'resultado_{nome}.png', img_visualizacao)

show_grid(resultados, titulos)
```

```
Aplicando h1...
Aplicando h2...
Aplicando h3...
Aplicando h4...
Aplicando h5...
Aplicando h6...
Aplicando h7...
Aplicando h8...
Aplicando h9...
Aplicando h10...
Aplicando h11...
```



Questão 2 - FFT

Funções engine para FFT

```
In [5]: def aplicar_fft(img):
    fft_original = np.fft.fft2(img)
    fft_centralizada = np.fft.fftshift(fft_original)
    return fft_centralizada

def aplicar_ifft(espectro_centralizado):
    fft_descentralizada = np.fft.ifftshift(espectro_centralizado)
    img_reconstruida = np.fft.ifft2(fft_descentralizada)
    return np.abs(img_reconstruida)
```

```

def calcular_espectro_visual(espectro):
    magnitude = np.abs(espectro)
    visualizacao = 20 * np.log(magnitude + 1)
    return visualizacao

def criar_grade_distancias(shape):
    linhas, colunas = shape
    centro_u, centro_v = linhas // 2, colunas // 2
    u, v = np.ogrid[:linhas, :colunas]
    distancias = np.sqrt((u - centro_u)**2 + (v - centro_v)**2)
    return distancias

def filtro_passa_baixa(shape, raio):
    dist = criar_grade_distancias(shape)
    return (dist <= raio).astype(np.float32)

def filtro_passa_alta(shape, raio):
    dist = criar_grade_distancias(shape)
    return (dist > raio).astype(np.float32)

def filtro_passa_faixa(shape, raio_interno, raio_externo):
    dist = criar_grade_distancias(shape)
    mascara = (dist >= raio_interno) & (dist <= raio_externo)
    return mascara.astype(np.float32)

def filtro_rejeita_faixa(shape, raio_interno, raio_externo):
    passa_faixa = filtro_passa_faixa(shape, raio_interno, raio_externo)
    return 1.0 - passa_faixa

def comprimir_por_fourier(espectro, percentual_corte=90):
    magnitude = np.abs(espectro)
    limiar = np.percentile(magnitude, percentual_corte)
    espectro_comprimido = np.copy(espectro)
    espectro_comprimido[magnitude < limiar] = 0
    return espectro_comprimido

def get_file_size(filepath):
    if os.path.exists(filepath):
        return os.path.getsize(filepath) / 1024
    return 0

```

Rodando

Filtros

```

In [6]: shape = img.shape
        espectro = aplicar_fft(img)

        cenarios = [
            {"nome": "1", "r": 15, "r_in": 5, "r_out": 25},
            {"nome": "2", "r": 60, "r_in": 30, "r_out": 80},
            {"nome": "3", "r": 150, "r_in": 100, "r_out": 200}
        ]

        for c in cenarios:
            r = c["r"]

```

```

r_in, r_out = c["r_in"], c["r_out"]

mascaras_cenario = {
    f"Passa-Baixa (r={r})": filtro_passa_baixa(shape, r),
    f"Passa-Alta (r={r})": filtro_passa_alta(shape, r),
    f"Passa-Faixa ({r_in}-{r_out})": filtro_passa_faixa(shape, r_in,
    f"Rejeita-Faixa ({r_in}-{r_out})": filtro_rejeita_faixa(shape, r

}

resultados_imgs = []
titulos_imgs = []

for nome, mascara in mascaras_cenario.items():
    espectro_filtrado = espectro * mascara

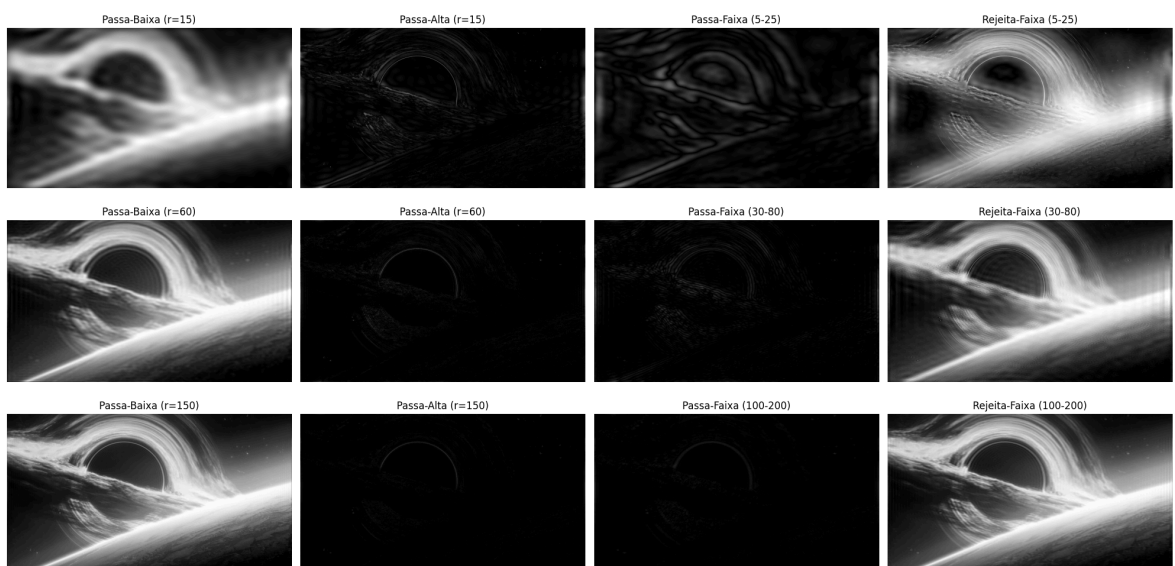
    img_reconstruida = aplicar_ifft(espectro_filtrado)

    resultados_imgs.append(img_reconstruida)
    titulos_imgs.append(nome)

    save_image(f"fourier_{c['nome']}_{nome.split(' ')[0]}.png", img_r

show_grid(resultados_imgs, titulos_imgs, cols=4)

```



Análise dos Filtros

A análise dos resultados obtidos nos três cenários de filtragem revela como a manipulação do raio de corte (r) e das bandas de frequência (r_{in} , r_{out}) impacta diretamente a reconstrução espacial da imagem do Gargantua.

Cenário 1 (Agressivo - Núcleos Curtos): Neste cenário, observou-se a maior perda de fidelidade visual. O filtro **Passa-Baixa** atuou de forma severa, mantendo apenas as frequências próximas ao centro, o que resultou em uma imagem excessivamente embaçada, onde o ruído de alta frequência foi eliminado, mas a definição do buraco negro foi perdida. O **Passa-Alta**, ao bloquear quase todo o centro, gerou uma imagem escura com bordas muito espessas. Uma percepção "psicodélica" foi percebida nos filtros **Passa-Faixa** e **Rejeita-Faixa**.

Cenário 2 (Moderado - Equilíbrio): Este cenário apresentou o melhor equilíbrio para a extração de características. No **Passa-Alta** e no **Passa-Faixa**, embora a imagem resultante seja naturalmente mais escura pela remoção das baixas frequências, brilho global, foi possível preservar e identificar o contorno e a silhueta do disco de acreção do Gargantua de forma mais nítida que nos demais cenários. Notou-se que o filtro **Rejeita-Faixa** neste estágio apresentou um comportamento visualmente similar ao Passa-Baixa do cenário anterior, pois ao rejeitar uma faixa intermediária considerável, ele acaba priorizando as frequências muito baixas, resultando em uma suavização controlada que ainda permite reconhecer a estrutura principal.

Cenário 3 (Suave - Alta Frequência): No cenário de núcleos amplos, as alterações foram sutis. O **Passa-Baixa** e o **Rejeita-Faixa** apresentaram resultados muito próximos à imagem original, indicando que a maior parte da energia da imagem está contida dentro do raio de 150 pixels. O **Passa-Faixa** e o **Passa-Alta**, por sua vez, isolaram apenas os detalhes mais finos e contornos afiados, com baixíssima presença de ruído ou artefatos de interferência. As silhuetas aqui são mais nítidas e "limpas", porém menos informativas sobre a estrutura volumétrica do objeto do que no Cenário 2, servindo primariamente para realce de detalhes finos.

```
In [7]: testes_percentual = [90, 95, 99, 99.9]
size_orig = get_file_size('gargantua_gray.png')

for p in testes_percentual:
    espectro_comprimido = comprimir_por_fourier(espectro, percentual_cort)
    img_comprimida = aplicar_ifft(espectro_comprimido)

    figure_name = f'compressao{p}.png'
    save_image(figure_name, img_comprimida)
    size_comp = get_file_size(OUT_DIR / figure_name)

    fig, axes = plt.subplots(1, 3, figsize=(20, 5))

    axes[0].imshow(img_comprimida, cmap='gray')
    info_texto = (f'Corte: {p}%\n'
                  f'Tamanho Original: {size_orig:.2f} KB\n'
                  f'Tamanho Comprimido: {size_comp:.2f} KB\n'
                  f'Economia: {100 - (size_comp/size_orig*100):.1f}%')

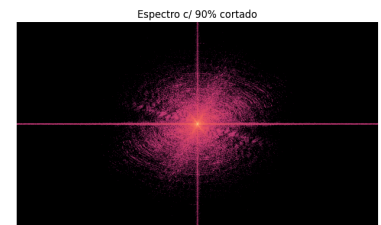
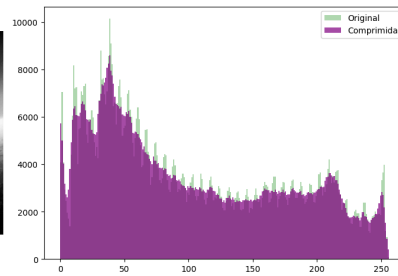
    axes[0].set_title(f'Gargantua Comprimido\n{info_texto}', fontsize=10,
                      axes[0].axis('off')

    # histograma comparativo
    axes[1].hist(img.ravel(), bins=256, range=[0,256], color='green', alp
    axes[1].hist(img_comprimida.ravel(), bins=256, range=[0,256], color='
    axes[1].legend()

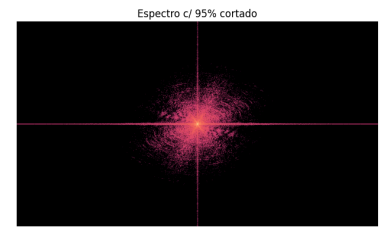
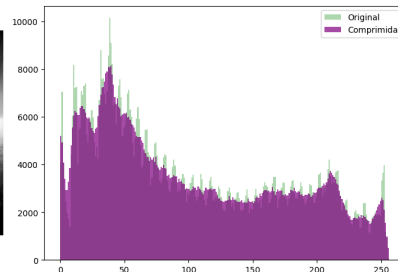
    esp_vis_comp = calcular_espectro_visual(espectro_comprimido)
    axes[2].imshow(esp_vis_comp, cmap='magma') # 'magma' fica top para es
    axes[2].set_title(f'Espectro c/ {p}% cortado')
    axes[2].axis('off')

    plt.tight_layout()
    plt.show()
```

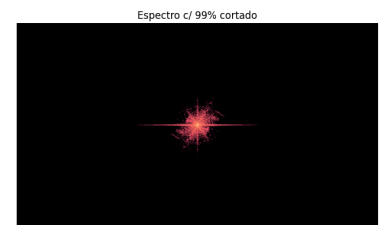
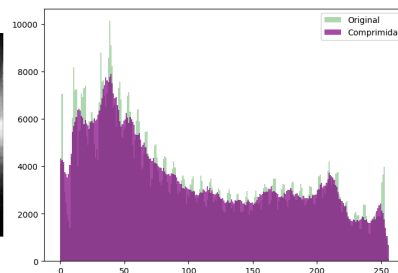

Gargantua Comprimido
Corte: 90%
Tamanho Original: 360.25 KB
Tamanho Comprimido: 379.91 KB
Economia: -5.3%



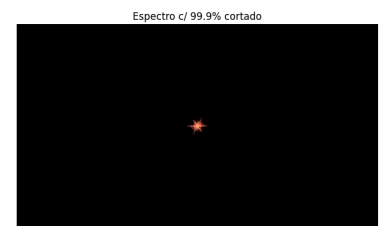
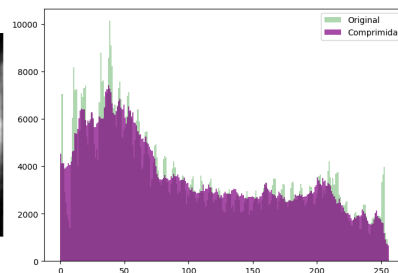
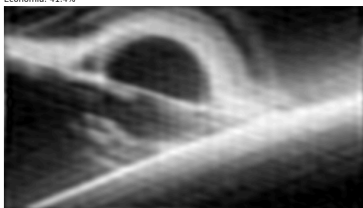
Gargantua Comprimido
Corte: 95%
Tamanho Original: 360.25 KB
Tamanho Comprimido: 352.89 KB
Economia: 2.0%



Gargantua Comprimido
Corte: 99%
Tamanho Original: 360.25 KB
Tamanho Comprimido: 294.16 KB
Economia: 18.3%



Gargantua Comprimido
Corte: 99.9%
Tamanho Original: 360.25 KB
Tamanho Comprimido: 211.27 KB
Economia: 41.4%



Análise da Compressão por Limiar de Magnitude

Nesta etapa, explorou-se a capacidade da Transformada de Fourier em compactar informações através do descarte de coeficientes de baixa magnitude. A técnica baseia-se no fato de que a maior parte da energia e das informações estruturais de uma imagem está concentrada em poucos coeficientes de alta magnitude, permitindo uma redução drástica de dados com perda mínima de percepção visual.

Metodologia de Comparação: Para validar a eficiência da compressão, utilizou-se uma abordagem comparativa em três frentes: a visualização do **espectro de magnitude logarítmica**, que revela a quantidade de coeficientes descartados; a análise de **histogramas de intensidade**, para medir o impacto na distribuição de tons de cinza; e a aferição do **tamanho dos arquivos em disco**, comparando o peso das imagens reconstruídas em relação ao arquivo original para quantificar a economia real de armazenamento.

Fidelidade Visual e Comportamento do Espectro: Observou-se que a quantidade de informação visual no espectro diminui bruscamente conforme aumentamos o

percentual de corte. Nos níveis de 90% e 95%, a imagem reconstruída do Gargantua permanece visualmente muito próxima à original, o que evidencia uma alta redundância de dados. Entretanto, a partir de 99%, a imagem apresenta um embaçamento severo. É notável que a diferença visual e de dados entre 99% e 99.9% é massiva, pois a compressão se torna extremamente agressiva no último nível, onde apenas os componentes mais fundamentais da imagem, são preservados.

Análise dos Histogramas: A comparação entre os histogramas revela que, conforme a compressão aumenta, o valor máximo das frequências de intensidade tende a diminuir e o delta, representando a variação de tons, fica visivelmente menor. Nas imagens mais comprimidas, como no caso de 99.9%, o histograma reflete a simplificação da imagem. Como há menos detalhes e variações bruscas, a distribuição de pixels se torna menos espalhada e mais concentrada em faixas específicas de cinza, o que confirma a perda de contraste e nitidez observada visualmente.

Eficiência de Armazenamento e a Curiosidade dos 90%: Um fenômeno interessante ocorreu no nível de 90% de corte, no qual o arquivo final apresentou um aumento de tamanho de aproximadamente 5.5% em relação ao original, enquanto em 95% a redução foi sutil, cerca de 2%. Isso acontece devido a uma característica técnica: ao zerar 90% dos coeficientes de Fourier, introduzimos pequenas oscilações matemáticas na imagem reconstruída, conhecidas como artefatos de *ringing*.

Para o olho humano, o resultado parece quase idêntico, mas para o algoritmo de armazenamento do arquivo PNG, essas micro-oscilações são interpretadas como ruído ou novos detalhes que precisam ser guardados com precisão. Como o PNG é um formato que não aceita perdas, ele acaba gastando mais bytes para descrever esse ruído matemático do que gastaria com a imagem original lisa. A economia real torna-se relevante apenas em níveis extremos: o ganho de 85 KB observado entre 90% e 99% foi praticamente igualado pelo salto de apenas 0.9%, de 99% para 99.9%, que economizou mais 83 KB. Isso demonstra que a compressão baseada em Fourier é extremamente poderosa para reduzir o peso de arquivos, desde que se aceite uma simplificação visual maior da imagem.